

VULNERABILITY ASSESSMENT s PENETRATION TESTING REPORT

Client : ABC Consulting

Target : <http://aspnet.testsparker.com/>

Assesment Period : 5 days

Conducted & Prepared by
Mayank Kumar Karn

1. Executive Summary -

This security test was carried out for **Nexavise Consulting** to check how secure the web application <http://aspnet.testsparker.com> really is. The test ran for about **50 hours** between **27 - 28 September** and included both manual checks and automated tools to look for weaknesses.

The goal was to find anything that could let an attacker break into the system, steal data, change information, or stop the application from working.

Several issues were found, with different levels of risk (**High, Medium, and Low**). If left unfixed, these could allow attackers to get access to sensitive data, take over user sessions, or even manipulate how the application works.

This report lists all the issues in detail, explains why they matter, and gives clear steps to fix them. The most serious problems should be fixed right away to reduce the chance of real-world attacks and improve the overall security of the application.

2. Scope -

Application Details (URL)	http://aspnet.testsparker.com
---------------------------	---

3. Approach s Methodology-

Our testing methodology was based on industry-standard frameworks like OWASP Testing Guide, NIST 800-115

We performed the following steps:

Reconnaissance s Enumeration

- Identified live hosts, open ports, and running services using **Nmap**.
- Collected subdomains and server information.
- Checked for publicly available files, backups, and metadata.

Automated Vulnerability Scanning

- Used tools like **Nikto** and **Burp Suite** for initial vulnerability discovery.
- Flagged issues like SQL Injection, XSS, and security misconfigurations.

Manual Verification s Exploitation

- Manually confirmed and safely exploited findings like **SQL Injection**, **Local File Inclusion**, and **Stored XSS** to demonstrate actual risk.
- Reviewed session handling, access controls, and business logic flaws.

Reporting s Risk Rating

- Each issue was assigned a severity (Critical/High/Medium/Low) based on potential business impact.
- Detailed recommendations were added for quick remediation.

4. Key Finding –

VULNERABILITY	SEVERITY
Sql injection on bitcoin calculator	Critical
Sql injection on product page	Critical
Local File Inclusion (LFI) on Help Page	Critical
Publicly Accessible Database File	High
FTP Logs Disclosure via WS_FTP.log	High
Stored Cross-Site Scripting (XSS) on Guestbook	High
Reflected Cross-Site Scripting (XSS) on About Page	High
EXIF Metadata Leak in Image File	Medium
Directory Listing Enabled on /statics/	Medium
Unauthorised product listing	Medium
Open Redirect on redirect.aspx	Medium
Session fixation	Medium
Sensitive Data Exposure via QR Code	Low

Risk Ratings -

- Each vulnerability is rated based on its impact and likelihood. We used a risk matrix to categorize the findings as follows:

Risk Rating	Description
Critical	Exploitable vulnerabilities that could lead to a complete system compromise.
High	Vulnerabilities that could lead to significant damage if exploited.
Medium	Issues that may lead to unauthorized access but require specific conditions.
Low	Minor vulnerabilities with limited potential for exploitation.

5. Information Gathering -

Initial recon was done to map the application and find potential entry points. This included DNS checks, port scanning, directory enumeration, and technology fingerprinting. Early issues like **directory listing**, **EXIF metadata leaks**, and **server path disclosure** were identified in this phase.

Nmap -

Command run - `sudo nmap -sS -sV -sC -p- -T4 aspnet.testsparker.com`

```

Nmap scan report for aspNet.testsparker.com (54.204.37.212)
Host is up (0.014s latency).
rDNS record for 54.204.37.212: ec2-54-204-37-212.compute-1.amazonaws.com
Not shown: 42865 filtered tcp ports (no-response), 22665 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp?
80/tcp    open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-title: Bitcoin Web Site
|_ http-robots.txt: 2 disallowed entries
|_ /statics/ /panel/
|_ http-generator: Bootply
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
443/tcp   open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-robots.txt: 2 disallowed entries
|_ /statics/ /panel/
|_ http-title: Bitcoin Web Site
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-generator: Bootply
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
554/tcp   open  rtsp?
1723/tcp  open  pptp?
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

```

sudo nmap -A -p 80,443 -T4 aspNet.testsparker.com

```

$ sudo nmap -A -p 80,443 -T4 aspNet.testsparker.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-28 15:49 IST
Nmap scan report for aspNet.testsparker.com (54.204.37.212)
Host is up (0.11s latency).
rDNS record for 54.204.37.212: ec2-54-204-37-212.compute-1.amazonaws.com

PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-generator: Bootply
|_ http-robots.txt: 2 disallowed entries
|_ /statics/ /panel/
|_ http-title: Bitcoin Web Site
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
|_ http-methods:
|_   Potentially risky methods: TRACE
443/tcp   open  http      Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header:
|_   Microsoft-HTTPAPI/2.0
|_   Microsoft-IIS/8.5
|_ http-methods:
|_   Potentially risky methods: TRACE
|_ http-robots.txt: 2 disallowed entries
|_ /statics/ /panel/
|_ http-title: Bitcoin Web Site
|_ http-generator: Bootply
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running (JUST GUESSING): Microsoft Windows 2012 (87%)
OS CPE: cpe:/o:microsoft:windows_server_2012:r2
Aggressive OS guesses: Microsoft Windows Server 2012 R2 (87%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 6 hops
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

TRACEROUTE (using port 80/tcp)
HOP RTT      ADDRESS
1  5.33 ms  jiofiber.local.html (192.168.31.1)
2  5.39 ms  192.0.0.1
3  ...
4  9.37 ms  192.0.0.1
5  11.94 ms  192.0.0.1
6  12.03 ms  ec2-54-204-37-212.compute-1.amazonaws.com (54.204.37.212)

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 28.42 seconds

```

sudo nmap -p 80,443 --script http-enum,http-title,http-server-header
aspnet.testsparker.com

```

PORT      STATE SERVICE
80/tcp    open  http
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ Microsoft-IIS/8.5
|_ http-title: Bitcoin Web Site
|_ http-enum:
|_ /clientaccesspolicy.xml: Microsoft Silverlight crossdomain policy
|_ /robots.txt: Robots file
|_ /crossdomain.xml: Adobe Flash crossdomain policy
443/tcp   open  https
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ Microsoft-IIS/8.5
|_ http-title: Bitcoin Web Site
|_ http-enum:
|_ /clientaccesspolicy.xml: Microsoft Silverlight crossdomain policy
|_ /robots.txt: Robots file
|_ /crossdomain.xml: Adobe Flash crossdomain policy
|_ /js/: Potentially interesting folder (401 Unauthorized)

Nmap done: 1 IP address (1 host up) scanned in 706.40 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-28 02:58 IST
Stats: 0:04:45 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 83.33% done; ETC: 03:04 (0:00:57 remaining)
Nmap scan report for ec2-54-204-37-212.compute-1.amazonaws.com (54.204.37.212)
Host is up (0.063s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Not Found
443/tcp   open  https
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Not Found

Nmap done: 1 IP address (1 host up) scanned in 1192.06 seconds

```

Nikto -

nikto -h <http://aspnet.testsparker.com>

```

(codemayank@mayhack)~$ nikto -h http://aspnet.testsparker.com
- Nikto v2.3.0

+ Target IP: 54.204.37.212
+ Target Hostname: aspnet.testsparker.com
+ Target Port: 80
+ Start Time: 2025-09-28 14:51:26 (GMT+5)

+ Server: Microsoft-IIS/8.5
+ /: Retrieved x-aspnet-version header: 4.0.30319.
+ /: Retrieved x-powered-by header: ASP.NET.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilit
ing-content-type-header/
+ /: Cookie testCookie created without the HttpOnly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /robots.txt: Entry /static/ is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/issues/00000000_robots-txt-file
+ /robots.txt: contains 2 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ /clientaccesspolicy.xml contains a full wildcard entry. See: https://docs.microsoft.com/en-us/previous-versions/windows/silverlight/dotnet-windows-silverlight/cc197955(vvs.95)?redirectedfrom=MSDN
+ /crossdomain.xml contains a full wildcard entry. See: http://jereemigrossman.blogspot.com/2008/03/crossdomain.xml-invites-cross-site.html
+ OPTIONS: Allowed HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST .
+ /sitemap.xml: This gives a nice listing of the site content.
+ /WS_FTP.LOG: WS_FTP.LOG file was found. It may contain sensitive information. See: OSVDB-13405

+ 8049 requests: 0 error(s) and 14 item(s) reported on remote host
+ End Time: 2025-09-28 16:00:58 (GMT+5) (4532 seconds)

```

Dirb -

```

(codemayank@mayhack)~$ dirb http://aspnet.testsparker.com/ /usr/share/wordlists/dirb/common.txt
DIRB v2.22
By The Dark Raver

START TIME: Sun Sep 28 15:53:39 2025
URL_BASE: http://aspnet.testsparker.com/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://aspnet.testsparker.com/ ---
=> DIRECTORY: http://aspnet.testsparker.com/administrator/
+ http://aspnet.testsparker.com/crossdomain.xml (CODE:200|SIZE:271)
+ http://aspnet.testsparker.com/dashboard (CODE:302|SIZE:156)
+ http://aspnet.testsparker.com/js (CODE:401|SIZE:2269)
=> DIRECTORY: http://aspnet.testsparker.com/panel/
+ http://aspnet.testsparker.com/robots.txt (CODE:200|SIZE:56)
+ http://aspnet.testsparker.com/sitemap.xml (CODE:200|SIZE:3221)
=> DIRECTORY: http://aspnet.testsparker.com/usercontrols/
+ http://aspnet.testsparker.com/WS_FTP.LOG (CODE:200|SIZE:771)

--- Entering directory: http://aspnet.testsparker.com/administrator/ ---
+ http://aspnet.testsparker.com/panel/
+ http://aspnet.testsparker.com/panel/.cv$ (CODE:503|SIZE:326)

--- Entering directory: http://aspnet.testsparker.com/usercontrols/ ---
=> DIRECTORY: http://aspnet.testsparker.com/usercontrols/pages/
=> DIRECTORY: http://aspnet.testsparker.com/usercontrols/Pages/

--- Entering directory: http://aspnet.testsparker.com/usercontrols/pages/ ---
+ http://aspnet.testsparker.com/usercontrols/pages/web.xml (CODE:503|SIZE:326)

--- Entering directory: http://aspnet.testsparker.com/usercontrols/Pages/ ---
=> Testing: http://aspnet.testsparker.com/usercontrols/Pages/title

END TIME: Sun Sep 28 17:55:12 2025
DOWNLOADED: 27672 - FOUND: 8

```

gobuster -

```

codermayank@mayhack: ~
└─$ gobuster dir -u http://aspenet.testsparker.com/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt -o gobuster.txt

gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:             http://aspenet.testsparker.com/
[+] Method:          GET
[+] Threads:         10
[+] Wordlist:         /usr/share/wordlists/dirbuster/directory-list-2.3-small.txt
[+] Negative Status codes: 404
[+] User Agent:       gobuster/3.6
[+] Timeout:         10s
[+] Retries:          10

Starting gobuster in directory enumeration mode

/3s      (Status: 404) [Size: 2260]
/dashboard      (Status: 402) [Size: 156]    => http://aspenet.testsparker.com/dashboard/
/panel      (Status: 301) [Size: 159]    => http://aspenet.testsparker.com/panel/
/administrator      (Status: 301) [Size: 167]    => http://aspenet.testsparker.com/administrator/
/25      (Status: 402) [Size: 2260]
Progress: 16726 / 87665 (19.08%)#ERROR Get "http://aspenet.testsparker.com/icon.9?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
Progress: 16765 / 87665 (19.08%)#ERROR Get "http://aspenet.testsparker.com/icon.10?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
ERROR Get "http://aspenet.testsparker.com/jsIcons?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
ERROR Get "http://aspenet.testsparker.com/404": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
ERROR Get "http://aspenet.testsparker.com/headers-end?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
ERROR Get "http://aspenet.testsparker.com/header_start?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
ERROR Get "http://aspenet.testsparker.com/eqid?": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
/dashboard      (Status: 302) [Size: 156]    => http://aspenet.testsparker.com/dashboard/
/UserControls      (Status: 301) [Size: 166]    => http://aspenet.testsparker.com/UserControls/
/status      (Status: 301) [Size: 163]    => http://aspenet.testsparker.com/status/
/UserControls      (Status: 301) [Size: 166]    => http://aspenet.testsparker.com/usercontrols/
Progress: 87664 / 87665 (100.00%)

Finished

```

6. Findings Details -

6.1 Sql injection on bitcoin calculator	Critical	CVSS: 3.1
OWASP Classification OWASP Top 10 2021: A03 — Injection		
Vulnerable URL / Parameter: http://aspnet.testsparker.com/Converter.aspx		
Description - The Converter.aspx page is vulnerable to error-based SQL Injection . The application takes user input from the btcAmount parameter and concatenates it directly into a SQL query without proper sanitization or parameterization. Supplying crafted input causes the database to return detailed SQL error messages and internal stack traces, confirming the injection point. This issue allows attackers to potentially enumerate database schema, read or modify data, and, depending on database permissions, escalate to full system compromise.		
Steps to reproduce - • Visit the target page: http://aspnet.testsparker.com/Converter.aspx • Enter any random value in the BTC field and click Calculate. • Capture the request using Burp Suite (or another proxy). • Locate the btcAmount parameter in the JSON request body. • Replace its value with the following payload: • ' OR 1=1—		

- Forward the request and observe the response. The server returns a SQL error message and stack trace, confirming that the input is being executed as part of a SQL query (SQL Injection vulnerability).

Impact -

- Attackers can **run malicious SQL queries**, read or modify sensitive data.
- Risk of **full database compromise** depending on permissions.
- **Internal file paths and stack traces** are exposed, helping attackers craft deeper exploits.

Remediation -

- Use **parameterized queries** or stored procedures for database operations.
- Validate input to allow **only numeric values** for btcAmount.
- Disable detailed error messages and return **generic error responses** in production.

Proof of Concept (poc)-

aspnet.testsparker.com/Converter.aspx

This website is automatically reset at every midnight (00:00 - UTC).

Bitcoin Web Site Home Blog Shop Converter & Pricings Demo Help Contact Login

Converter

Please enter the BTC amount you want to calculate. Then, click to Calculate Button.
(Does not reflect the truth.)

(Example: 1234,56 BTC = 1234,56)

1 BTC

Calculate

298,78 USD

Pricings

If you see Pricings, Click here.

burp suite community edition v2.3.12 - temporary project

Dashboard Target Proxy Intruder Help Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x 2 x 3 x 4 x 5 x 6 x +

Send Cancel < >

Target: http://aspnet.testparker.com HTTP/1.1

Request

```
1 POST /ConverterResponse.aspx HTTP/1.1
2 Host: aspnet.testparker.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: application/json, text/javascript, */*; q=0.01
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/json; charset=utf-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 28
10 Origin: http://aspnet.testparker.com
11 Cookie: ASP.NET_SessionId=9yv2hl4kxaj5Gv1qr
12 Referer: http://aspnet.testparker.com/Converter.aspx
13 Priority: u=0
14 {
15   "btcAmount": "1" OR 1=1
16 }
```

Response

```
1 HTTP/1.1 200 OK
2 Cache-Control: private
3 Content-Type: application/json; charset=utf-8
4 Server: Microsoft-IIS/8.5
5 X-AspNet-Version: 4.0.30319
6 X-Powered-By: ASP.NET
7 Date: Sun, 28 Sep 2020 12:26:42 GMT
8 Content-Length: 2161
9
10 Unquoted quotation marks after the character string 'or'1=1--
11
12 Incorrect syntax near 'or'1=1--
13
14 at System.Data.SqlClient.SqlConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction)
15 at System.Data.SqlClient.TdsParser.ThrowExceptionAndWarning(TdsParserStateObject stateObj, Boolean callerHasConnectionLock, Boolean asyncClose)
16 at System.Data.SqlClient.TdsParser.TryRun(RunBehavior runBehavior, SqlCommand commandHandler, SqlDataReader readerStream, BulkCopySimpleResultSet bulkCopyHandler, TdsParserStateObject stateObj, Boolean isAdapted)
17 at System.Data.SqlClient.SqlDataReader.TryConsumeMetaData()
18 at System.Data.SqlClient.SqlCommand.FinishExecuteReader(SqlDataReader ds, RunBehavior runBehavior, String optionsString, Boolean isInternal, Boolean forDescribeParameterEncryption, Boolean shouldCachePerfOnlyWayEncrypted)
19 at System.Data.SqlClient.SqlCommand.RunExecuteReaderTds(SqlCommandBehavior behavior, Boolean returnStream, Boolean async, Boolean asyncClose)
```

Done 2,389 bytes (1,361 milli)

6.2 Sql injection on product page

Critical

CVSS v3.1: 9.8

OWASP Classification

OWASP Top 1 2021: A03 — Injection

Vulnerable URL / Parameter: <http://aspnet.testsparker.com/Products.aspx?pld=1>

Description -

The pld parameter in the product page is directly concatenated into a SQL query without validation or parameterization. Supplying crafted input causes SQL syntax errors and error messages to be returned in the application response. During testing, SQLmap and manual payloads confirmed that the query could be manipulated, proving the presence of SQL Injection. This vulnerability allows attackers to interact with the backend database and exfiltrate or alter sensitive data.

Steps to Reproduce -

- Open the following URL with a malicious value for pld
<http://aspnet.testsparker.com/Products.aspx?pld=-1' AND 1=2-->
- The application responds with HTTP 200 but includes a SQL error in the page, for example: **Incorrect syntax near '1'**
- This confirms the injected SQL was executed by the backend.

Impact -

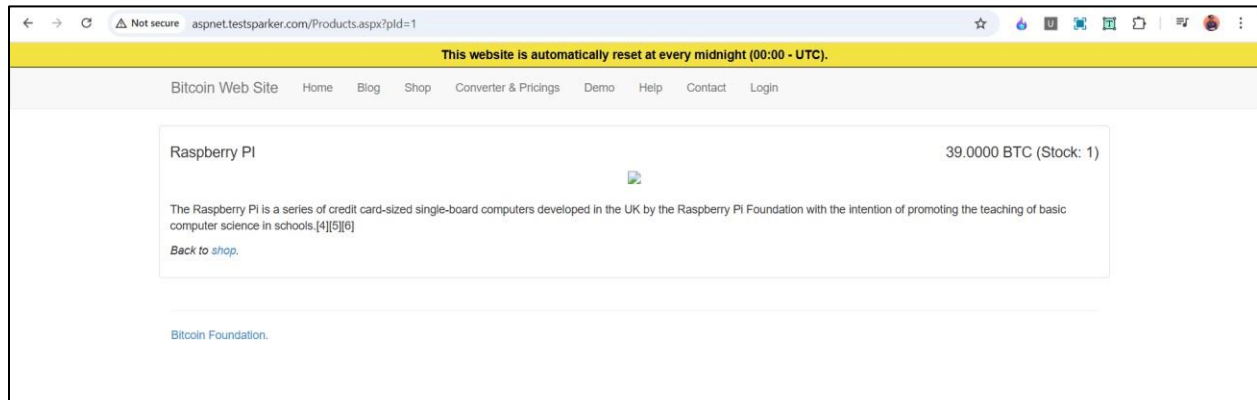
- Attackers can execute **arbitrary SQL queries** on the backend.
- Possible outcomes include **user data disclosure, order manipulation, and privilege escalation** depending on database permissions.
- Error messages returned in the HTML response leak database details and file paths, making exploitation easier.
- If leveraged fully, this vulnerability could lead to a **complete database compromise**.

Remediation -

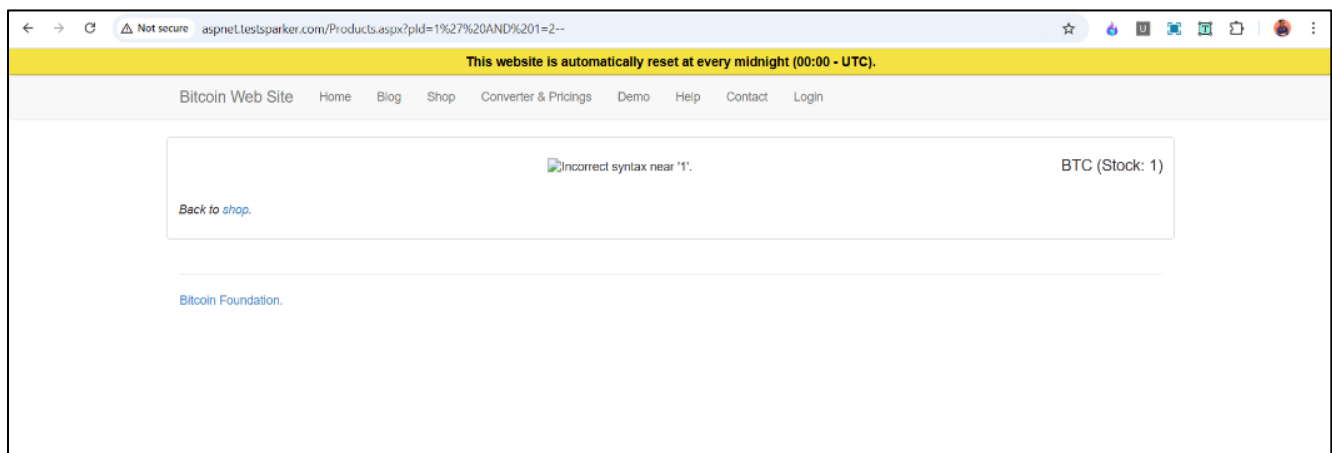
- Replace dynamic SQL with **parameterized queries or prepared statements**.
- Enforce strict input validation: only allow numeric values for pld (e.g., `int.TryParse()` in .NET).
- Suppress detailed database error messages — use **generic error pages** and log full details server-side.
- Ensure the database account used by the application follows **least-privilege principles** (no admin/dbo rights).
- Deploy **WAF rules** temporarily to detect and block SQLi payloads until fixes are applied.
- Retest the application after remediation to confirm the issue is resolved.

Proof of Concept (poc) -

Normal view



after adding malicious value



6.3 Local File Inclusion (LFI) on Help Page

Critical

CVSS v3.1: 8.8

OWASP Classification: CWE-22 / Path Traversal (A05:2021 – Security Misconfiguration / Sensitive Data Exposure)

Vulnerable URL / Parameter: <http://aspnet.testsparker.com/Help.aspx?item=1>

Description -

The item parameter in Help.aspx is vulnerable to **Local File Inclusion (LFI)**. The application reads files from disk based directly on user-supplied input without validating or sanitizing the path. By injecting ../ directory traversal sequences, attackers can retrieve arbitrary files from the server, such as win.ini, web.config, or application source files.

Steps to Reproduce -

- Open the Help page: <http://aspnet.testsparker.com/Help.aspx?item=1>
- Replace the item parameter value with a directory traversal payload: <http://aspnet.testsparker.com/Help.aspx?item=../../../../Windows/win.ini> or we use curl command too
- The server responds with the contents of the win.ini file, confirming the vulnerability..

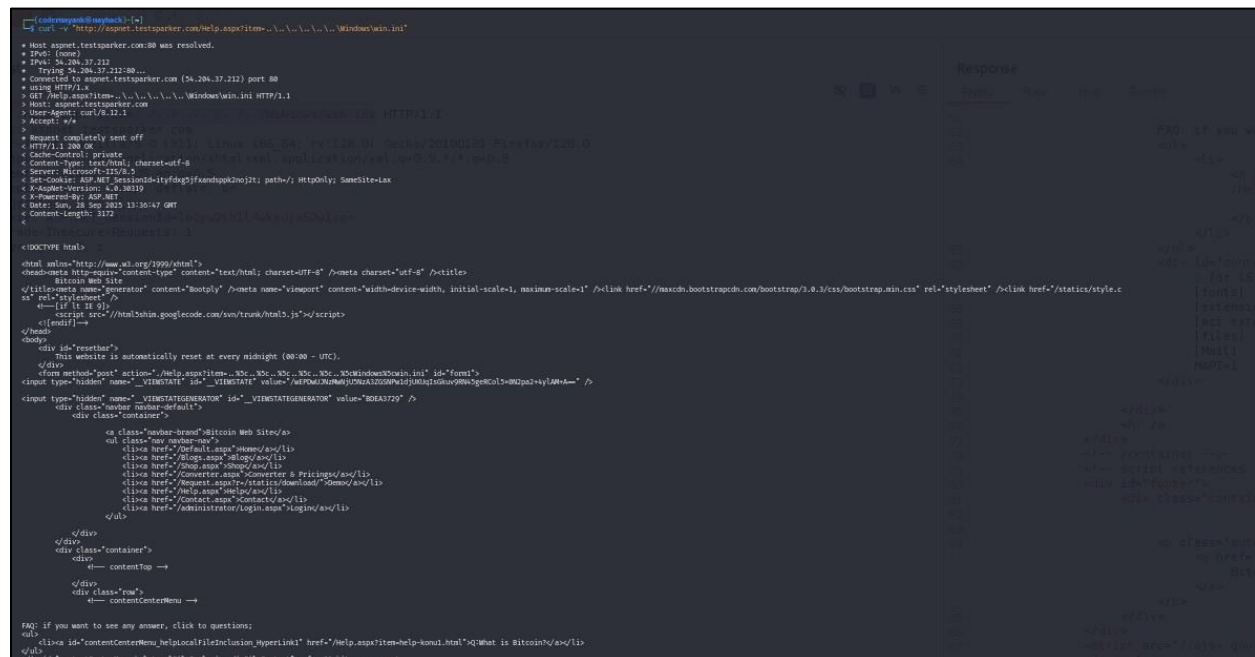
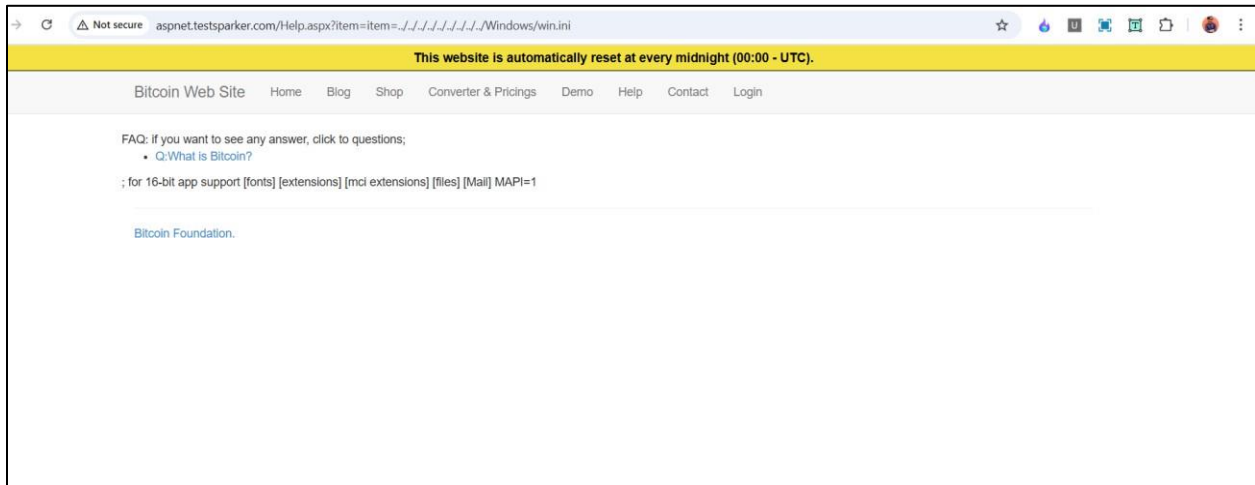
Impact -

- Attackers can read sensitive configuration files, including database connection strings, API keys, or system credentials.
- Disclosure of source code may reveal further vulnerabilities (hard-coded passwords, logic flaws).
- Provides reconnaissance material for privilege escalation or remote code execution if writeable files or logs are discovered.

Remediation -

- Stop using raw filenames from user input. Use IDs mapped to allowed help files.
- Canonicalize and validate paths: resolve the full path (Path.GetFullPath) and ensure it stays within a predefined safe directory. Reject input containing ../, : or encoded traversal sequences.
- Allowlist permitted files and block all others (return 403/404).
- Run the web application under a least-privileged account with no access to OS-level sensitive files.
- Add logging and monitoring for traversal attempts and test regularly for LFI.

Proof of Concept -



6.4 Publicly Accessible Database File

High

CVSS v3.1: 7.5

OWASP Classification: A05:2021 – Security Misconfiguration

Vulnerable URL: <http://aspnet.testsparker.com/statics/data.mdb>

Description –

The Microsoft Access database file data.mdb is directly accessible from the web root. Anyone who knows or discovers the URL can download the file and access its contents offline. During testing, the file was manually downloaded and opened, revealing stored application data, including personal identifiers.

Steps to Reproduce -

- Open the following URL directly in a browser: <http://aspnet.testsparker.com/statics/data.mdb>
- The file downloads automatically.
- Open the .mdb file in Microsoft Access (or compatible viewer).
- Confirm that real application data (including names and other records) is visible.

Impact -

- Sensitive Data Exposure: Attackers can extract all records from the database, including user information, product data, or configuration values.
- Data Integrity Risk: A malicious actor could alter and re-upload data in certain misconfigured environments.
- Provides reconnaissance material that could be used for further attacks such as authentication bypass, privilege escalation, or phishing.

Remediation -

- **Move the database file outside of the web root** so it cannot be downloaded over HTTP.
- Restrict direct access via web server configuration (`web.config` or `.htaccess` rules).
- Encrypt sensitive fields within the database to limit damage if compromised.
- Regularly review server for **exposed backup or data files**.
- Consider migrating from an Access database to a more secure, server-side RDBMS if appropriate.

Proof of Concept(poc) -



6.5 FTP Logs Disclosure via WS_FTP.log

high

CVSS v3.1: 7.5

OWASP Classification: A05:2021 – Security Misconfiguration

Vulnerable URL: http://aspnet.testsparker.com/WS_FTP.log

Description -

The WS_FTP.log file is publicly accessible on the web server. This log file contains detailed FTP activity, including uploaded/downloaded file names and potentially user account information. Allowing public access to such logs exposes sensitive operational data and could aid attackers in planning further attacks.

Steps to Reproduce -

- Open the following URL directly in a browser:

http://aspnet.testsparker.com/WS_FTP.log
- Observe that the server returns the contents of the FTP log file in plain text, listing past file transfers and server activity...

Impact -

- **Information Disclosure:** Reveals file names, paths, timestamps, and possibly usernames or IPs involved in FTP transfers.
- May expose **sensitive files** uploaded to the server or disclose structure of directories.
- Provides reconnaissance data that can be used for **targeted exploitation** of known files or directories.

Remediation -

- Remove public access to log files from the web root.
- Store logs in a non-web-accessible directory with proper file permissions.
- If logs must be accessed through the web, enforce authentication and authorization controls.
- Regularly review logs to ensure no sensitive data (passwords, PII) is being stored in plain text.
- Update server configuration to block directory listing and direct log access.

Proof of Concept(poc) -



```
103.05.06 13:17 B d:\smz\notes\colors.xml <-- mathsmz /smz/notes colors.xml
103.05.06 13:33 B D:\SMZ\notes\colors.xml --> duh.xtech.ru /SMZ/notes colors.xml
103.05.06 13:33 B D:\SMZ\notes\dwSiteColumnsMe.xml --> duh.xtech.ru /SMZ/notes dwSiteColumnsMe.xml
103.05.06 13:33 B D:\SMZ\notes\Flash.xml --> duh.xtech.ru /SMZ/notes flash.xml
103.05.06 13:33 B D:\SMZ\notes\images.xml --> duh.xtech.ru /SMZ/notes images.xml
103.05.06 13:33 B D:\SMZ\notes\movies.xml --> duh.xtech.ru /SMZ/notes movies.xml
103.05.06 13:33 B D:\SMZ\notes\scripts.xml --> duh.xtech.ru /SMZ/notes scripts.xml
103.05.06 13:33 B D:\SMZ\notes\shockwave.xml --> duh.xtech.ru /SMZ/notes shockwave.xml
103.05.06 13:33 B D:\SMZ\notes\urls.xml --> duh.xtech.ru /SMZ/notes urls.xml
```


6.6 Stored Cross-Site Scripting (XSS) on Guestbook

high

CVSS v3.1: 7.4

OWASP Classification: A03:2021 – Injection

Vulnerable URL / Parameter: <http://aspnet.testsparker.com/Guestbook.aspx> – input fields for name/message

Description –

The Guestbook feature fails to sanitize user-supplied input before storing and displaying it. Malicious payloads, such as an tag with an onerror event handler, are stored in the database and executed when other users view the guestbook entries. This allows attackers to execute arbitrary JavaScript in victims' browsers in a persistent manner.

Steps to Reproduce -

- Open the Guestbook submission page: <http://aspnet.testsparker.com/Guestbook.aspx>
- Enter the following payload in the input field(s):
- Submit the form.
- Open the Guestbook listing page: <http://aspnet.testsparker.com/GuestbookList.aspx>
- Observe that the payload executes, showing a JavaScript alert — confirming stored XSS.

Impact -

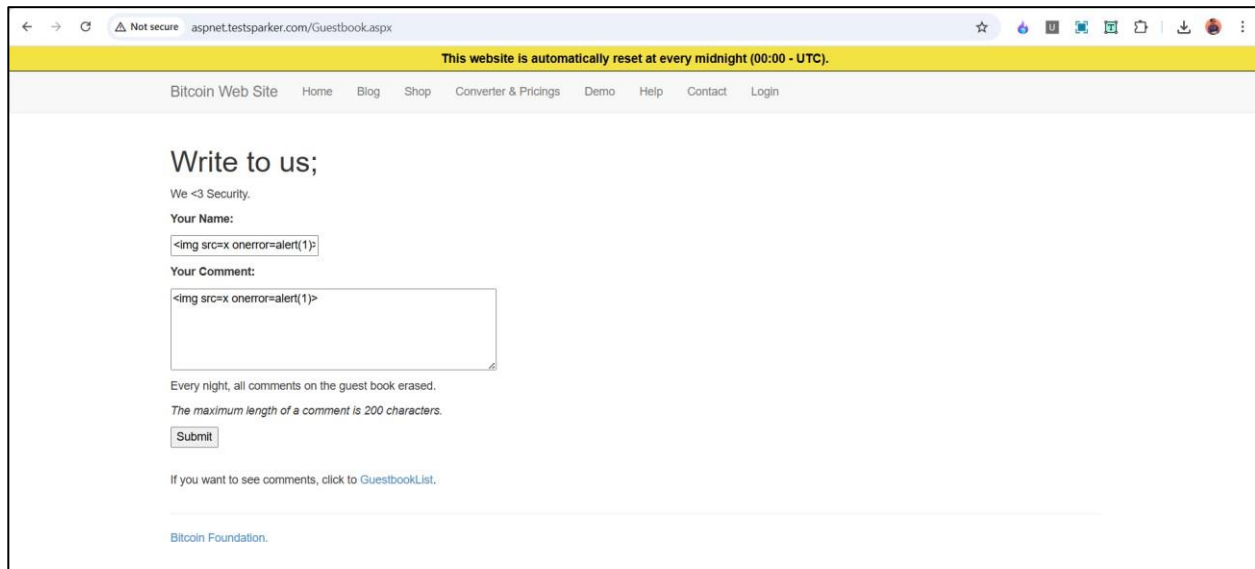
- **Session Hijacking: Steal session cookies and impersonate users.**
- **Credential Theft / Phishing: Inject fake login prompts or malicious links.**
- **Defacement / Malicious Scripts: Modify content shown to visitors or spread malware.**

Remediation -

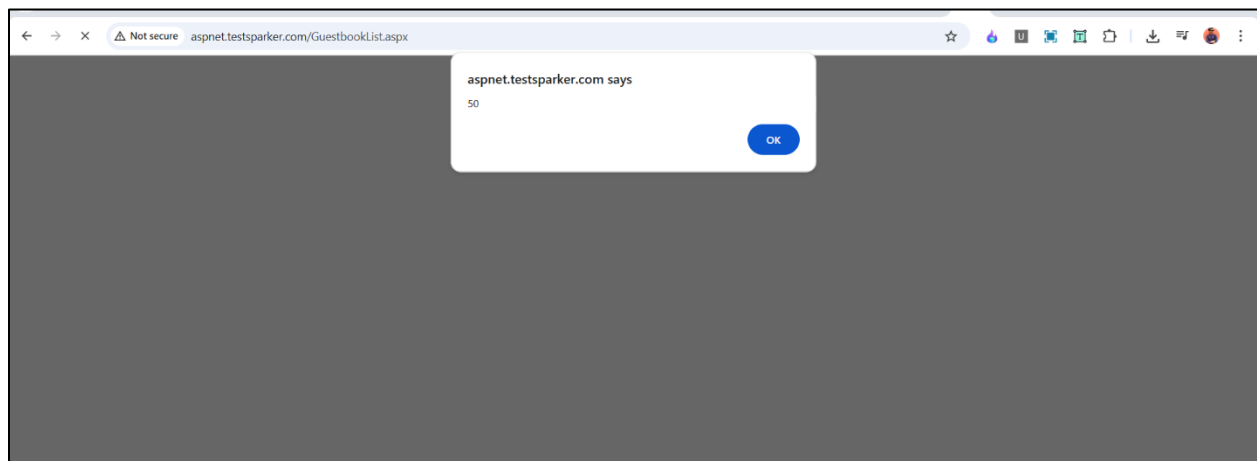
- Apply proper output encoding (HTML-encode user input before rendering it).
- Use a server-side input validation library to reject malicious payloads (strip <script> tags, event handlers, etc.).
- Consider using Content Security Policy (CSP) headers to reduce XSS impact.
- Escape dynamic content in templates using built-in encoding functions (e.g., `HttpUtility.HtmlEncode` in ASP.NET).

Proof of Concept(POC) -

Putting xss payload in input box -



Then in guestbooklist we can confirm payload of stored xss



6.7 Reflected Cross-Site Scripting (XSS) on About Page

Medium

CVSS v3.1: 6.1

OWASP Classification: A03:2021 – Injection

Vulnerable URL / Parameter: <http://aspnet.testsparker.com/About.aspx?hello=1> → [parameter: hello](#)

Description -

The hello parameter in About.aspx is vulnerable to **reflected Cross-Site Scripting (XSS)**. User input is returned directly in the response without proper HTML encoding, allowing malicious JavaScript to be executed in the victim's browser. This issue can be exploited by crafting a malicious link and tricking a victim into clicking it, resulting in the execution of arbitrary JavaScript in their session.

Steps to Reproduce -

- Open the vulnerable page with the crafted payload:

http://aspnet.testsparker.com/About.aspx?hello=

- Load the URL in a browser.
- Observe that the alert box pops up showing the document's domain name, confirming execution of attacker-supplied JavaScript..

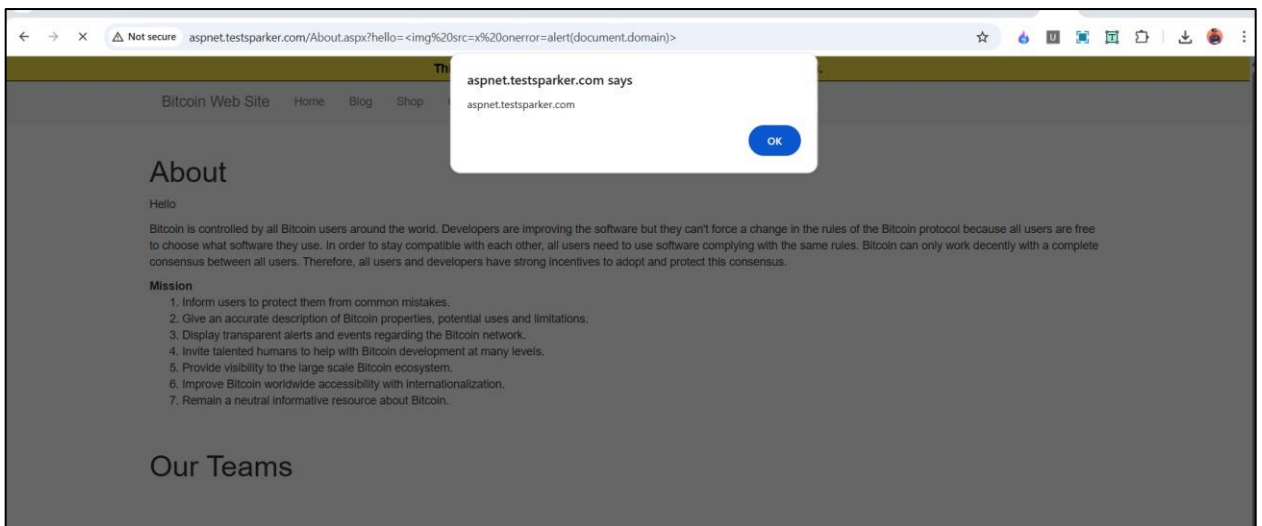
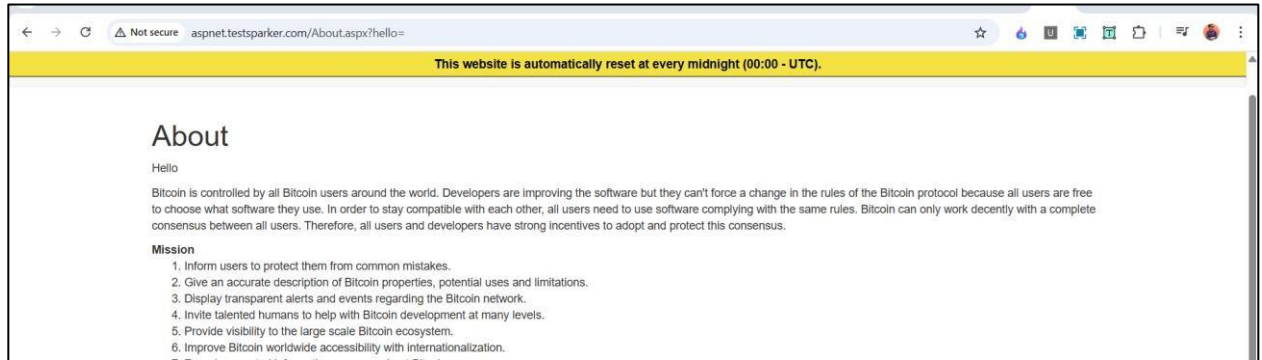
Impact -

- **Session Theft:** Attackers can steal session cookies if victims click malicious links.
- **Phishing / Social Engineering:** Inject fake login prompts or redirect users to malicious sites.
- **Reputation Risk:** Could be used to deface the page or spread malicious payloads through links.

Remediation -

- Encode all user-supplied input before rendering it in the response (use HTML encoding).
- Implement input validation to reject or sanitize dangerous characters (<, >, ", ', on*= attributes).
- Consider implementing Content Security Policy (CSP) headers to mitigate XSS risk.
- Test all input points during development to ensure proper encoding is in place.

Proof of Concept (poc)-



6.8 EXIF Metadata Leak in Image File

Medium

CVSS v3.1: 5.3

OWASP Classification: A05:2021 – Security Misconfiguration

Vulnerable URL: <http://aspnet.testsparker.com/statics/write-us.jpg>

Description -

The write-us.jpg image contains **EXIF metadata** that is publicly accessible. Analysis of the file using an online EXIF viewer revealed the ProfileDateTime field, which discloses the exact creation or modification date of the image. While not a direct code execution risk, such information can aid attackers in fingerprinting content update cycles, deployment timelines, or identifying outdated components.

Steps to Reproduce -

- Open the image directly:

<http://aspnet.testsparker.com/statics/write-us.jpg>
- Download the file and upload it to an online EXIF metadata viewer or in terminal exiftool
- Review the metadata fields — note the ProfileDateTime field revealing the timestamp of creation/modification.

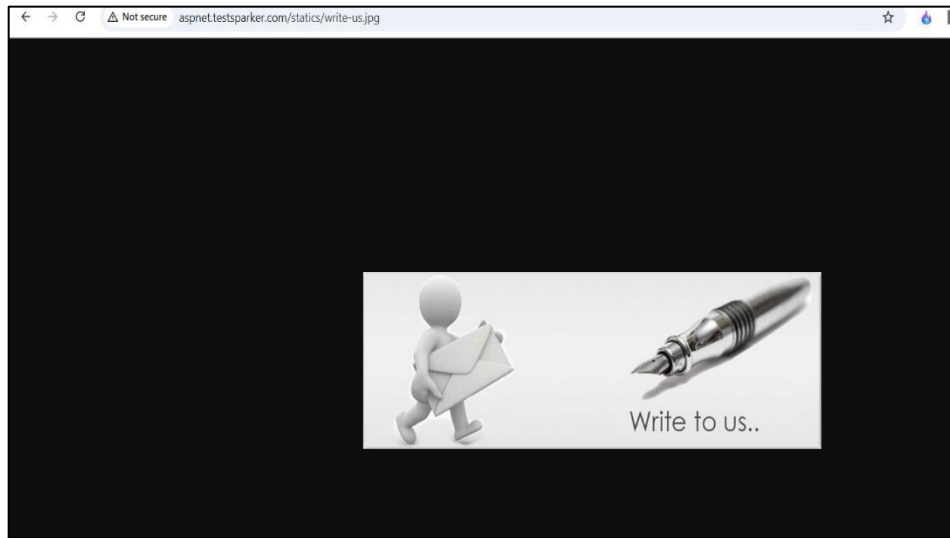
Impact -

- Information Disclosure: Reveals internal timestamps that could assist attackers in mapping update schedules or identifying stale/unused files.
- Could indirectly support social engineering or timing-based attacks.
- In some cases, EXIF data can contain usernames, software versions, or geolocation if not stripped

Remediation -

- Strip EXIF metadata from images before uploading them to production servers (use tools like exiftool or automated image sanitization in CI/CD).
- Automate metadata removal as part of the deployment pipeline.
- Review existing media files for sensitive data (e.g., creator names, GPS info) and re-upload sanitized versions.

Proof of Concept(poc) -



```
al $ exiftool write-us.jpg
Exiftool Version Number      : 13.10
File Name                   : write-us.jpg
Directory                   :
File Size                   : 25 kb
File Modification Date/Time  : 2025:09:28 20:15:51+05:30
File Access Date/Time       : 2025:09:28 20:15:51+05:30
File Inode Change Date/Time : 2025:09:28 20:15:51+05:30
File Permissions            : -rw-rw-r--
File Type                   : JPEG
File Type Extension         : jpg
MIME Type                   : image/jpeg
JFIF Version                : 1.01
Resolution Unit             : inches
X Resolution                : 192
Y Resolution                : 192
Exif Byte Order             : Big-endian (Motorola, MM)
Pixel Units                 :
Pixels Per Unit X           : 2835
Pixels Per Unit Y           : 2835
Profile CMM Type            : Linotronic
Profile Version             : 2.1.0
Profile Class               : Display Device Profile
Color Space Data            : RGB
Profile Connection Space    : XYZ
Profile Date Time           : 1998:02:09 06:49:00
Profile File Signature      : acsp
Primary Platform            : Microsoft Corporation
CMM Flags                   : Not Embedded, Independent
Device Manufacturer        : Hewlett-Packard
Device Model               : sRGB
Device Attributes          : Reflective, Glossy, Positive, Color
Rendering Intent            : Media-Relative Colorimetric
Connection Space Illuminant : 0.9642 1 0.82491
Profile Creator             : Hewlett-Packard
Profile ID                 : 0
Profile Copyright          : Copyright (c) 1998 Hewlett-Packard Company
Profile Description         : sRGB IEC61966-2.1
Media White Point           : 0.95045 1 1.08905
Media Black Point          : 0 0 0
Red Matrix Column           : 0.43607 0.32249 0.01392
Green Matrix Column         : 0.34915 0.71687 0.09708
Blue Matrix Column          : 0.14307 0.00001 0.71411
Device Mfg Desc             : IEC http://www.iec.ch
Device Model Desc          : IEC 61966-2.1 Default RGB colour space - sRGB
Viewing Cond Desc           : Reference Viewing Condition in IEC61966-2.1
Viewing Cond Illuminant    : 19.6445 20.3718 16.8089
Viewing Cond Surround      : 3.52589 4.07439 3.36179
Viewing Cond Illuminant Type : D50
Luminance                  : 76.03647 80 87.12462
Measurement Observer       : CIE 1931
Measurement Backing        : 0 0 0
Measurement Geometry       : Unknown
Measurement Flare          : 0.999%
Measurement Illuminant     : D65
Technology                 : Cathode Ray Tube Display
Red Tone Reproduction Curve : (Binary data 2060 bytes, use -b option to extract)
Green Tone Reproduction Curve : (Binary data 2060 bytes, use -b option to extract)
Blue Tone Reproduction Curve : (Binary data 2060 bytes, use -b option to extract)
Image Width                : 603
Image Height               : 200
Encoding Process           : Baseline DCT, Huffman coding
Bits Per Sample            : 8
Color Components            : 3
Y Cb Cr Sub Sampling       : YCbCr4:2:0 (2 2)
Image Size                 : 603x200
Megapixels                 : 0.121
```

6.G Directory Listing Enabled on /statics/

Medium

CVSS v3.1: 5.3

OWASP Classification: A05:2021 – Security Misconfiguration

Vulnerable URL: <http://aspnet.testsparker.com/statics/>

Description -

The /statics/ directory on the server has **directory listing enabled**, allowing anyone to view the internal file and folder structure. This reveals all files hosted under this directory, some of which may contain sensitive data (e.g., data.mdb, WS_FTP.log). This information could be leveraged by attackers to find and download exposed resources or identify further attack vectors.

Steps to Reproduce -

- Open the directory URL:
`http://aspnet.testsparker.com/statics/`
- The server responds with an auto-generated directory index showing all files in the folder.

Impact -

- Information Disclosure: Exposes file names and folder structure to unauthenticated users.
- Makes it easier for attackers to locate sensitive files such as backups, logs, and configuration data.
- Increases risk of exploitation when combined with other vulnerabilities (e.g., exposed .mdb file was discovered via this listing)

Remediation -

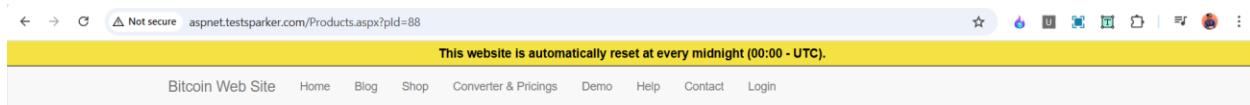
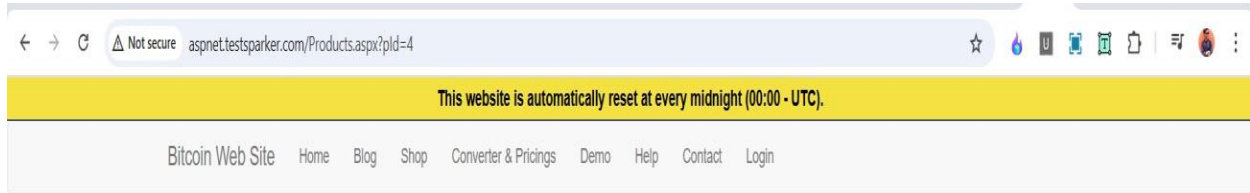
- Disable directory listing in the web server configuration (web.config for IIS, Options -Indexes for Apache).
- Ensure only explicitly linked files are publicly accessible.
- Move sensitive files outside the web root or restrict access using proper authorization controls.
- Periodically review public folders for exposed files and remove unnecessary ones.

Proof of Concept(poc) -



6.10 Unauthorized Product Listing	Medium	CVSS v3.1: 6.5
OWASP Classification: A01:2021 – Broken Access Control		
Vulnerable URL / Parameter: http://aspnet.testsparker.com/Products.aspx?pld=1 → parameter: pld		
Description - The Products.aspx endpoint is vulnerable to Insecure Direct Object Reference (IDOR) . The pld parameter is used directly to fetch product details without proper authorization checks. By sequentially modifying the value of pld in the URL, attackers can enumerate all product records and access information they may not be authorized to view.		
Steps to Reproduce - • Navigate to the shop page : http://aspnet.testsparker.com/Shop.aspx • View a product, then capture the request in Burp Suite. • Identify the product request URL: <code>/Products.aspx?pId=1</code> • Manually change pId in the query string to other values: <code>/Products.aspx?pId=2</code> <code>/Products.aspx?pId=3</code> • Observe that product details for other IDs are returned without authentication or authorization checks.		
Impact - • Information Disclosure: Attackers can view details of all products by simply iterating through IDs. • Could lead to competitive intelligence leakage if sensitive product data (pricing, SKUs) is exposed. • May serve as a starting point for other attacks (e.g., SQLi, price manipulation if update endpoints exist).		
Remediation – • Implement proper authorization checks before returning product details. Ensure the user is allowed to view the requested resource. • Do not rely solely on sequential IDs for access control. Use randomized or indirect identifiers (GUIDs or hashed IDs). • Apply rate-limiting and monitoring to detect enumeration attempts. • Include logging and alerts when unusual access patterns are detected.		

Proof of Concept (poc)-



[Bitcoin Foundation.](#)

6.11 Open Redirect on redirect.aspx

Medium

CVSS v3.1: 6.1

OWASP Classification: A01:2021 – Broken Access Control / A10:2021 – Server-Side Request Forgery (SSRF)

Vulnerable URL : <http://aspnet.testsparker.com/redirect.aspx?site=bitcoin.com>

Description -

The site parameter in redirect.aspx is vulnerable to **open redirect**. The application takes user-supplied input and redirects the browser to any specified URL without validating whether it is an allowed domain. Attackers can exploit this by crafting malicious links that appear to originate from the legitimate site but redirect users to phishing pages or malicious downloads.

Steps to Reproduce -

- Open the following URL in a browser:
<http://aspnet.testsparker.com/redirect.aspx?site=https://evil.com>
- The browser is immediately redirected to <https://evil.com>.
- Replace evil.com with any attacker-controlled site to demonstrate arbitrary redirection.

Impact -

- Enables phishing attacks by making malicious links look like they come from a trusted site.
- Can be used to bypass security filters or reputation checks (URL appears safe initially).
- May allow chaining with other attacks (e.g., redirecting users to an exploit kit page).

Remediation -

- Validate the site parameter against an allowlist of trusted domains.
- Reject or strip dangerous input (e.g., absolute URLs pointing outside the application's domain).
- Consider using relative redirects within the same domain only.
- Display an interstitial warning page before redirecting users to external sites.

6.12 Session Fixation	Medium	CVSS v3.1: 6.5
OWASP Classification: A02:2021 – Cryptographic Failures / A07:2021 – Identification & Authentication Failures		
Vulnerable Component: <u>ASP.NET SessionId cookie on http://aspnet.testsparker.com/administrator/Login.aspx</u>		
Description - The application is vulnerable to session fixation . The ASP.NET_SessionId cookie value is accepted even after login, meaning an attacker can preset a session value for a victim and then hijack their session after successful authentication. Proper session handling should issue a new session ID upon login to prevent session hijacking attacks.		
Steps to Reproduce - • Navigate to the login page: http://aspnet.testsparker.com/administrator/Login.aspx?r=/Dashboard/ • Observe the ASP.NET_SessionId cookie value in the browser (Developer Tools → Application/Storage). • Manually change the cookie value to a random string with the same length (e.g., AAAAAAAAAAAAAAAAAA) • Log in with valid credentials. • After login, check the cookie again — the session ID remains the same as the attacker-set value, confirming session fixation..		
Impact - • Enables phishing attacks by making malicious links look like they come from a trusted site. • Can be used to bypass security filters or reputation checks (URL appears safe initially). • May allow chaining with other attacks (e.g., redirecting users to an exploit kit page).		
Remediation - • Regenerate session IDs upon authentication and privilege elevation events. • Invalidate old session tokens immediately after login. • Configure ASP.NET to issue a new ASP.NET_SessionId after successful login (using Session.Abandon() and re-issue). • Use secure session cookie flags: HttpOnly, Secure, and SameSite=Strict. • Monitor for abnormal session reuse and add logout on session change detection.		

Proof of Concept (poc) -

This website is automatically reset at every midnight (00:00 - UTC).

Switch to Authentication v2 (Bot protection)

Email Address(alan@turing.com)

Password(theturingtest)

Sign In

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition K...	Cross Site	Priority
ASP.NET_SessionId	aaaaaaaaaaaaaa	aspnettes...	/	Session	33	✓		Lax			Medium
TestCookie	Hello	aspnettes...	/	Session	15						Medium

This website is automatically reset at every midnight (00:00 - UTC).

Hello Admin

You entered successfully.

Logout

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition K...	Cross Site	Priority
ASP.NET_SessionId	aaaaaaaaaaaaaa	aspnettes...	/	Session	29	✓		Lax			Medium
TestCookie	Hello	aspnettes...	/	Session	15						Medium

6.13 Sensitive Data Exposure via QR Code

low

CVSS v3.1: 3.7

OWASP Classification: A05:2021 – Security Misconfiguration / Sensitive Data Exposure

Vulnerable URL / File: <http://aspnet.testsparker.com/statics/qrcode.png>

Description -

A QR code image hosted under /statics/ reveals a Bitcoin wallet address when scanned. While this is not a direct vulnerability, it may unintentionally disclose internal or test data (e.g., a developer wallet address or placeholder information). Attackers may leverage this for phishing attempts, social engineering, or reconnaissance.

Steps to Reproduce -

- Open the QR code file: <http://aspnet.testsparker.com/statics/qrcode.png>
- Scan the QR code using any QR reader (mobile device or online scanner).
- The decoded content reveals a Bitcoin address, e.g.:
1Lj6e8dQRASKBv9vAyrytW1JLaenvGXZgP

Impact -

- **Public disclosure of a cryptocurrency address could expose business wallet information or be used for fraudulent solicitations.**
- **Attackers may impersonate the company, send fake payment requests, or trace blockchain transactions.**

Remediation -

- Remove unnecessary QR codes from the public web root if they are not required.
- If legitimate, replace with a company-controlled address and ensure it is documented.
- Use staging/test environments for testing data and avoid exposing test assets publicly..

Proof of Concept (poc) -

